



Edge Hill University

Faculty of Arts and Science

The Department of Computer Science

CIS1703

Programming 2

Coursework 2 (CW2)

2025/2026

Level 4

Tomas Striska – 26590191

Joshua Lemaire – 26625415

Architecture Design

System Overview

UniScheduler system is a resource booking system for scheduling university facilities such as meeting rooms and computer laboratories. The main purpose of the system is to provide users with a method to create bookings in an organised way while ensuring important constraints such as time and room capacity are enforced.

In a university environment manual room booking can result in errors such as double booking or overbooking rooms. This system addresses this problem by automating the process and providing immediate feedback to the user. When booking a room, the system checks whether the room is already booked at that time and whether the capacity is valid. In case of detection of any problem the booking is rejected.

It also saves booking data in JSON files so that the information is preserved even after the program is closed. This is to allow users to come back to the system and still see the bookings they made before. In conclusion the system offers a simple but effective solution for managing resources in a structured and reliable manner.

Class Structure and UML Design

There are many classes in the system, and each one is in charge of a different part of the program. This makes it easier to keep track of the code and manage it.

The Resource class is the base class and has shared properties like resource ID location and capacity. This class stands for any kind of resource that can be booked in the system.

The Resource class has two subclasses: LabSpace and Meeting Room. These subclasses add their own unique properties to the base class. For example, LabSpace might show how many computers are available and Meeting Room might show whether a projector is available. This lets the system show different kinds of rooms while still having some things in common.

The Booking class is used to show a single booking. It keeps track of things like who made the reservation, when it was made, what resource was chosen and how many people are coming. This class helps you organise booking data in a way that makes sense.

The Scheduler class is in charge of the system's overall logic. It can add bookings, check for time conflicts, make sure there is enough space and give you booking information when you need it. It is the main controller for the system.

The UML diagram shows these relationships in a picture. It shows that Meeting Room and LabSpace are both subclasses of Resource and that the Scheduler works with both the Booking and Resource classes. This diagram shows how the system is set up and how its parts work together.

Application of Object-Oriented Principles

Abstraction

The system uses abstraction to make things like rooms and bookings easier to understand. The system doesn't try to deal with every possible detail. Instead, it only looks at the most important ones like capacity, location and booking time. This makes the system easier to understand and cuts down on extra complexity.

The system lets developers work with higher-level ideas instead of low-level implementation details by using abstraction. This makes it easier to read and adds to the system in the future.

Inheritance

The Resource base class is what makes inheritance possible. This class is the parent of both LabSpace and Meeting Room. This means that they have the same properties as capacity and location.

This stops the same code from being used more than once and makes sure that all resource types are the same. For instance, the base class defines a capacity value for both types of rooms so they will always have one.

Inheritance also helps the system grow. You can add a new type of resource like a lecture hall by creating a new subclass of Resource without changing the code that is already there.

Encapsulation

Encapsulation is a way to protect data in each class and limit who can access it. Instead of letting people directly access variables, the system uses methods to safely handle data.

The Resource class has a method called `get capacity ()` that lets you get to capacity. This makes sure that values are processed correctly and that bad data is not used.

Encapsulation makes the system more reliable by reducing the chance of errors and keeping data organised within each object.

Polymorphism

Polymorphism is implemented by the subclasses overriding the methods. The Resource class defines a generic method for example `get_details()`, and each subclass overrides this method with its own implementation.

For example, LabSpace may return details on the number of computers while Meeting Room may return information about a projector. The behaviour changes depending on the object type even though the method name is the same.

This means the system can treat different kinds of resources in a uniform way but can also behave in a distinct way. It also makes the code more flexible and easier to extend.

Design Justification

The overall design of the system was chosen due to its clarity organization and suitability for the requirements of the project. This breaks the system into different classes where each part of the program has a specific responsibility. This makes the code easier to understand and maintain.

Object-Oriented Programming makes the system modular meaning changes can be made to one part of the system without affecting other parts. It is also especially useful for debugging and future development.

The design is also very scalable. In the existing structure you can make minimal changes to add new features or resource types. For example you could add new room types or booking features by extending the existing classes.

In general, the design that was chosen strikes a good balance between being simple and useful. It meets the requirements for the course and shows important programming ideas like inheritance encapsulation and polymorphism.

Implementation and Robustness

Core Functionality

Users can make look at and save reservations for university resources with this system. Adding bookings checking room availability checking capacity and saving data so that bookings are not lost when the program closes are some of the key features.

Conflict Detection

Conflict detection stops two people from booking the same resource at the same time. The system checks to see if the room and time the user chose are already booked when they make a reservation. The booking is turned down if there is a match. This is important because it stops people from booking the same thing twice and makes the system more trustworthy.

Capacity Validation

Capacity validation is used to check if the selected room can hold the number of people given by the user. Each resource has a limit to how much it can contain. If the user enters a number larger than the room can accommodate the system rejects the booking. This prevents invalid reservations and adds more realism to the system.

Defensive Programming

The system is written using defensive programming principles to minimise errors and crash. To make sure that users give appropriate values input validation is used (e.g. for capacity numbers). Error handling is also used so if something goes wrong the program does not crash. For example, if a file cannot be found then the system can return an empty list instead of crashing.

Data Persistence

The system saves the booking data as JSON files. When the program opens it loads existing booking data from the file. When the user exits the modified list of bookings is written back into the file. So even after the program is closed reservations can still be made.

Testing and Edge Cases

Test Case	What Was Tested	Expected Result
Double booking	Same room and time entered twice	Booking is rejected
Invalid capacity	User enters text instead of number	Error message shown
Capacity exceeded	Too many people entered	System starts safely
Empty bookings	User views bookings before adding any	Message shows no bookings
Missing JSON file	Program runs without data file	System starts safely

Human-Computer Interaction

Interface Design

The system has both a command-line interface (CLI) and a graphical user interface (GUI) that was made with Tkinter. The CLI version has a simple menu-based layout where users choose options by typing in numbers. You can add a booking see your bookings or leave the system as your main options. This structure makes it easy to understand and interact with.

The GUI version makes it easier to use by replacing text-based input with visual elements like labels text fields buttons and a list display. Users can type in information about their reservations and click buttons to do things. The list box shows all the bookings in one clear place. This makes the system easier to use especially for people who are not used to using command-line tools because they do not have to remember commands.

Usability Principles

To make the overall user experience better the system design follows a few important usability rules.

Giving users immediate feedback after they do something makes the system's status visible. For instance, a confirmation message shows up when a booking is successfully

added. If a booking does not go through because of a conflict or lack of space an error message will show up. This makes sure that users always know what is going on.

Input validation and system checks are used to stop errors from happening. Users cannot enter bad data into the system like putting text instead of numbers for capacity. It also checks for time conflicts and capacity limits before accepting input which keeps users from making invalid bookings.

The interface stays consistent throughout. The layout prompts and messages all have a similar structure which makes it easier for users to figure out how to use the system. Buttons and input fields are arranged in a logical way in the GUI which makes it easier to use.

Allowing users to move around the system freely supports user control. Users can go back to the main menu at any time in the CLI version. Users can choose actions with buttons in the GUI version without having to follow a strict order of steps.

There are clear and helpful messages that help with error recovery. If there is an error like when the user enters the incorrect information the system tells them what went wrong and lets them try again. This stops people from getting angry and makes it easy for them to fix mistakes.

Evaluation of Interface

There are a lot of good things about the interface. The CLI version is simple lightweight and easy to set up so it is good for quick testing and interaction. The GUI version makes it easier to use by adding visual elements that make it more user-friendly which lowers the chance of user error.

The system also gives clear feedback which helps users figure out what their actions mean. The system is more reliable and easier to use because it uses validation and error messages.

There are however a few weaknesses. The GUI is simple it does not have advanced design features like improved layout colours or responsive design. That means it may not appear as professional as commercial apps. The CLI version also requires the users to manually type in the inputs which may not be so intuitive for beginners.

Overall, the interface is functional consistent and easy to use and meets the project requirements. The system offers a robust usable booking management interface although there is some room for improvement particularly in visual design.

Collaboration and Development Process

Team Structure

The project was finished by two people with each person taking on different tasks to make sure that development went smoothly and tasks were organised clearly.

Each member was assigned specific roles based on their contributions to the project:

- Member 1: Tomas Striska
Responsible for:
 - Core system logic (Scheduler)
 - Conflict detection and capacity validation
 - Backend functionality
 - Testing and debugging
- Member 2: Joshua Lemaire
Responsible for:
 - User interface (CLI and/or GUI using Tkinter)
 - File handling and JSON data storage
 - Backend functionality
 - Testing and debugging

Both members helped with writing reports making documentation and designing the whole system including the UML diagram.

To speed up the process tasks were split up so that they could be worked on at the same time. Both members talked to each other regularly to make sure that all the parts worked together properly and that any problems were fixed quickly.

This way of working together made sure that both members worked on all the important parts of the system and that the project was finished well.

Version Control

The main version control system for the UniScheduler project was GitHub. It helped both team members work together well by letting them share code keep track of changes and handle updates during the development process.

The repository was set up at the beginning of the project, and both members were given permission to add to it. The team split up the work and each person made commits to the repository as they finished their tasks. This made sure that progress was recorded on a regular basis instead of all at once.

We used to commit to keep track of changes to the project like adding new features fixing bugs and making the code better. Each commit had a short message that explained what changes were made which made it easier to understand how the system had changed over time.

GitHub also helped put various parts of the system together. Version control let the two members combine their work without overwriting each other's changes because they were both working on different files. This made it less likely that work would be lost and made working together easier.

GitHub also served as a backup for the project. All the code was stored online so it could be recovered if any local files were lost or damaged.

In general, using GitHub helped people work together stay organised and be responsible. It made it clear who did what and made sure that the project was done in a professional and organised way.

Project Management

(The project was handled with a mix of simple planning and constant communication between both team members. With the short deadline it was important to organise tasks clearly and to focus on the most key features first.

The main requirements were identified at the start of the project for example; implementing a booking system applying Object Orientated Programming principles and making the system robust and useable. Both members split up the tasks so development could happen in parallel.

During the development process meetings were held on an informal basis face-to-face or on-line. These meetings were held to discuss the progress identify issues and decide the next steps. This helped ensure that both members were on the same page and that the project was progressing smoothly.

Planning was focused on finishing the most important parts first like making bookings finding conflicts and storing data. Once these features were working the focus moved to making the system easier to use testing edge cases and making it better.

Version control with GitHub also helped with project management because it let both members keep track of changes and updates to the code. This made it easier to work together and keep things from getting messy when putting different parts of the system together.

The project was managed in a simple but effective way that made sure all the important tasks were finished on time and that team members could work together and stay organised.

Challenges and Solutions

The development of the UniScheduler system was not without its challenges. One of the biggest challenges was dealing with time management with the deadline being so tight. Due to the brief time of the project, it was important to focus on key features such as booking creation conflict detection and data storage. The answer to this challenge was to start with the basic functionality and gradually improve the system.

Another challenge was making sure the booking logic worked correctly especially the conflict detection. First there were cases where bookings that should have been rejected were accepted. This was fixed by refining the logic in the Scheduler class and testing it against several scenarios to ensure time clashes were handled correctly.

Collaboratively there were some difficulties too. There were occasional minor problems coordinating the tasks and integrating the code from the two team members for example inconsistencies in coding style or small integration issues. These were ironed out through regular communication and review of each other's work before finalising the changes.

Also, it was difficult to handle the user input especially in terms of invalid data. If this check is not performed properly the system may crash or start behaving unpredictably. To solve this, we used input validation and try/except blocks to safely handle errors.

In general, these challenges were overcome with planning testing and communication which helped to ensure the system was completed successfully.

Reflection

Working on the UniScheduler system was a fantastic way to learn how to program and work with others. One of the most important things we learned was how important it is to plan before you start coding. It was easier to add features in a logical and organised way once the system structure was laid out and the most important classes were found.

The project also helped people learn more about Object-Oriented Programming. Using ideas like inheritance encapsulation and polymorphism in a real project helped me remember what I had learned in theory and showed me how these ideas can be used to make systems that are easy to maintain and well-organised.

Another important thing I learned was how important it is to test and debug. Finding and fixing bugs especially in booking validation and user input handling showed how important it is to write code that works and think about edge cases.

The project showed how important communication and working together are when it comes to teamwork. To work together everyone had to know what their job was and get regular updates to make sure that all the parts of the system worked together correctly.

If the project were to be done again more time would be spent making the user interface better and adding new features like a search function or a better booking system. But overall, the project was a success and gave a good base in software development practices.

System Evaluation and Further Discussion

Strengths of system

The UniScheduler system has a lot of good points especially when it comes to how it is set up and how it works. One of the best things about it is that it clearly follows Object-Oriented Programming rules. The code is easier to read and understand because the system is split into classes like Resource Booking and Scheduler. The system is also easier to keep up with and add to in the future because of its modular design.

Another big plus is the use of validation logic. The system does a decent job of stopping bad bookings by checking for conflicts and capacity. This makes sure that bookings are real and that two people cannot always book the same resource. This feature makes the system more useful and shows that it can do more than just store data.

Using defensive programming techniques also makes the system strong. When wrong data is entered input validation and error handling keep the program from crashing. Also using JSON to store data makes sure that booking information is saved and can be used again in future sessions.

Another good thing about it is that it has both a command-line interface (CLI) and a graphical user interface (GUI). The CLI is a simple and light way to use the system while the GUI makes it easier to use by making it more user-friendly. This shows that you understand the needs of different users and makes the app superior overall.

Weaknesses

The system has some problems even though it works well. The user interface is one of the main problems. The GUI makes it easier to use but it is still basic and does not have any advanced design features like better layout styling or responsiveness. This means that it might not be as good as more professional apps.

Another problem is that the booking system does not do much. Right now, the system only lets you make basic bookings. It does not let you do more advanced things like recurring bookings editing existing bookings or filtering/searching bookings. These features would make the system more useful in the real world.

In some parts of the implementation the system also depends on hardcoded resource data. This makes things less flexible because you must change the code instead of just updating a data source to add new resources. A more advanced system would save resources in a file or database that changes over time.

Performance discussion

The system works well for small-scale use which is what the project needs. A list holds the booking data and operations like finding conflicts are done by going through this list again. This method is quick and easy to use for a small number of bookings.

But as more people book performance may go down. This is because the system uses a linear search to look for conflicts which means that the time it takes to check for conflicts goes up as more bookings are made. This could make things run slower in bigger systems.

Using more efficient data structures or algorithms could make things work better. For instance, sorting bookings by time or using indexed data structures could make it faster to check for conflicts.

Design limitations

The system's current design works well for a small-scale application, but it will not work as well if the application needs to grow in the future. One problem is that JSON is used to store data. JSON is great for small projects, but it is not the best choice for handling a lot of data or many users. A database system would work better for bigger programs.

Another problem is that there is no way to verify users. The system does not tell users apart so anyone can make reservations without having to prove who they are. User accounts and permissions would be needed in a real-world system to keep things safe and hold people accountable.

The system also does not have any advanced scheduling features like automatically resolving conflicts or making the best use of bookings. These features would need more complicated algorithms, but they would make the system work much better.

The design works well for the current needs, but it would need to be expanded and improved to work with a larger and more realistic application.

Future Improvements

For improvements in the future, what would be expected is to increase the quality of the Graphical user Interface to make it more professional and allow people to use it simultaneously. Furthermore we would also use a database such as marinaDB to allow live usage and for people to be able to constantly use it in real time and it to update.

Conclusion

The UniScheduler system does a fantastic job of meeting its goal of offering an easy and useful way to book resources. Users can make and manage bookings with the system which makes sure that important rules like room capacity and time conflicts are followed. The system gives a dependable way to organise and keep track of university resources by adding these features.

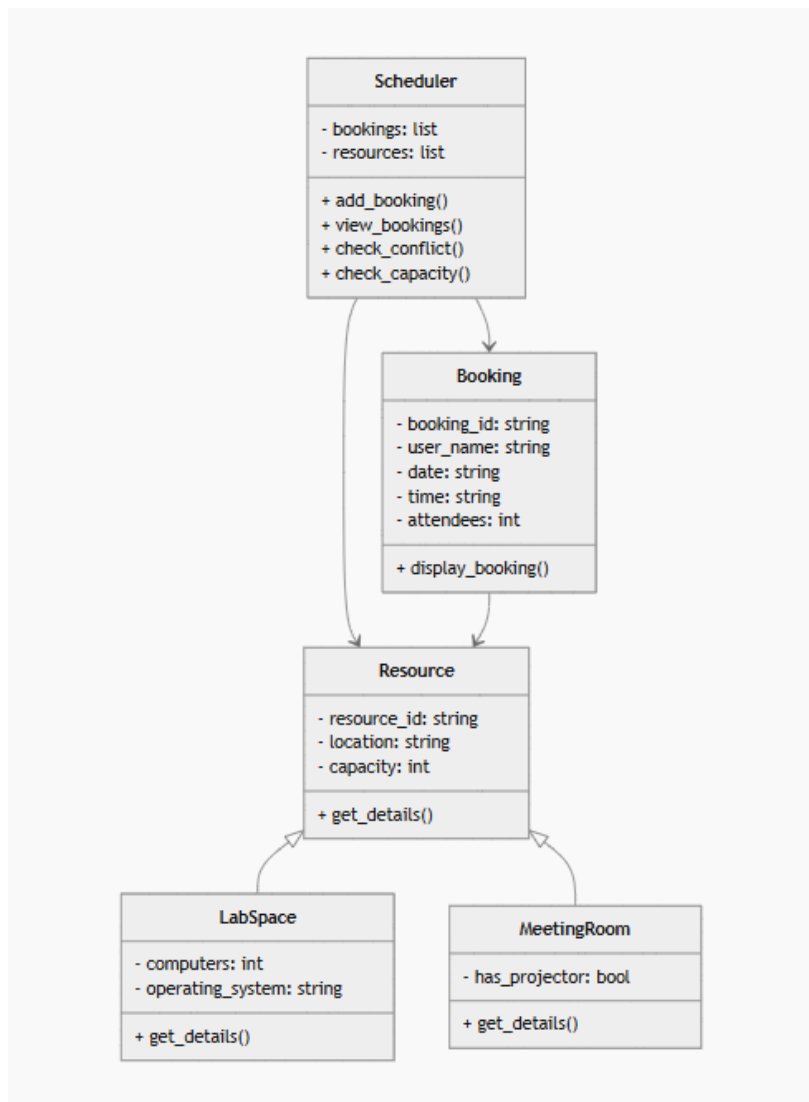
The project fulfils the essential criteria of the coursework by employing Object-Oriented Programming principles such as abstraction inheritance encapsulation and polymorphism. Structured classes like Resource Booking and Scheduler show how these ideas work in practice. The system also meets the requirements for robustness by using JSON files for validation error handling and data persistence.

The system also makes it easier to use by having both a command-line interface and a graphical user interface. This makes it easier for people to use the system and interact with it. The system's reliability is even better because it includes testing and handling of edge cases.

In general, the UniScheduler system can be thought of as a successful implementation. It meets the project's functional and technical needs and shows that the person understands the principles of software design and development. The current system does a decent job of meeting the goals of the coursework even though there are areas that could be better.

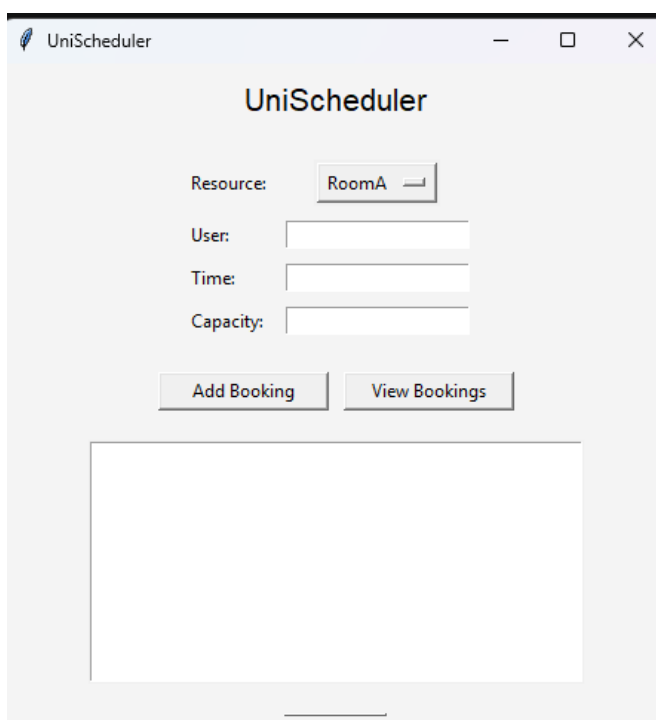
Appendix

UML diagram showing system class structure and relationships:



- **Screenshots of System**

```
--- UniScheduler ---  
1. Add booking  
2. View bookings  
3. Exit  
Choose option: 1  
Enter room (RoomA / Lab1): RoomA  
Enter your name: Josh  
Enter time (e.g 10:00): 10:00  
Enter number of people: 20  
Booking successful.
```



The screenshot shows a window titled "UniScheduler" with a light gray background. At the top, the title bar includes a feather icon, the text "UniScheduler", and standard window controls (minimize, maximize, close). The main content area has the title "UniScheduler" centered at the top. Below the title, there are four input fields with labels: "Resource:" (with a dropdown menu showing "RoomA"), "User:", "Time:", and "Capacity:". Below these fields are two buttons: "Add Booking" and "View Bookings". At the bottom of the window is a large, empty rectangular box, likely intended for displaying booking details or a list of bookings.